Number Theory Workshop

Problem 1

Let n be a positive integer. Show that all divisors of $4n^2+1$ have the form $4k+1$ for some integer k.

Solution 1

Let p be a prime divisor of $4n^2+1$. Then

$$(2n)^2 \equiv -1 \pmod{p}$$

so -1 is a quadratic residue mod p, giving $\left(\frac{-1}{p}\right) = 1$. By 1st complement,

$$\left(\frac{-1}{p}\right) = (-1)^{\frac{p-1}{2}} = 1$$

which forces $\frac{p-1}{2}$ to be even, and hence $p \equiv 1 \pmod{4}$. Since every divisor of $4n^2+1$ is a product of such primes, and a product of integers of the form $4k+1$ remains of the form $4k+1$, all divisors of $4n^2+1$ have the form $4k+1$.

Problem 2

Let p be a positive odd prime number with positive integers a and b such that $a^2+b^2=p$ where a is an odd number. Prove that there exists an integer n such that n^2 and a has the same remainder when divided by p.

Solution 2

It suffices to show that every prime divisor of a is a quadratic residue mod p, since products of quadratic residues are again quadratic residues.
Let q be an arbitrary prime divisor of a. Since $a^2+b^2=p$, reducing mod q gives

$$b^2 \equiv p \pmod{q},$$

so p is a quadratic residue mod q, i.e. $\left(\frac{p}{q}\right) = 1$.
Since a is odd and $a^2 < p$, both a and b are strictly less than $\sqrt{p}$, and since $a^2+b^2=p$ with a odd, one checks that $p \equiv 1 \pmod{4}$. By quadratic reciprocity, since $p \equiv 1 \pmod{4}$,

$$\left(\frac{q}{p}\right)\left(\frac{p}{q}\right) = 1,$$

and therefore

$$\left(\frac{q}{p}\right) = \left(\frac{p}{q}\right) = 1.$$

Since every prime factor q of a is a quadratic residue mod p, it follows that a itself is a quadratic residue mod p.

Problem 3

Find the first three smallest, non-negative solutions to $2^x \equiv 2 \pmod{11}$

Solution 3

For a power of 2 to have a remainder of 2 when divided by 11, $2^x = 11 \cdot M + 2$ where M is an integer. For the first case, note that $2^1 = 11 \cdot 0 + 2$ is $2 \pmod{11}$ hence, $x = 1$ is a valid solution.

Now recall Fermat's little theorem $a^{p-1} \equiv 1 \mod p$ where p is a prime and a is an integer. By this theorem, $2^{11-1} \equiv 1 \pmod{11}$ and by extension $2^{11-1} \cdot 2^{11-1} \equiv 1 \cdot 1 \pmod{11}$. Consequently, multiplying both equations by 2 we have $2 \cdot 2^{11-1} \equiv 2 \cdot 1 \pmod{11}$, $2 \cdot 2^{11-1} \cdot 2^{11-1} \equiv 2 \cdot 1 \cdot 1 \pmod{11}$ which results in $2^{11} \equiv 2 \pmod{11}$, $2^{21} \equiv 2 \pmod{11}$.

Hence $x = 11, 21$ are also solutions and note that by continually applying Fermat one could observer that $x \equiv 1 \pmod{11}$.

Problem 4

Find the least odd prime factor of $2026^{30} + 1$.

Solution 4

Let p be that prime, hence we know $2026^{30} + 1 \equiv 0 \pmod{p}$.

Noting that, $(2026^{15})^2 + 1^2 \equiv 0 \pmod{p}$, we can apply Fermat's Theorem on the Sum of two squares to conclude $p \equiv 1 \pmod{4}$. So the possible values for prime p are $5, 13, 17, \ldots$. Now we simply need to test which p causes the above equation to be true, which can be re-expressed as $2026^{30} \equiv -1 \pmod{0}$.

Now with $p = 5$:

$$\begin{aligned} 2026^{30} &\equiv 1^{30} \pmod{5} \quad (1 \text{ is remainder of } 2026/5) \\ 1^{30} &\equiv 1 \pmod{5} \end{aligned}$$

Hence, $p = 5$ is not a valid solution.
Testing $p = 13$:

$$\begin{aligned} 2026^{30} &\equiv 11^{30} \pmod{13} \quad (11 \text{ is remainder of } 2026/13) \\ 11^{30} &\equiv (11^2)^{15} \pmod{13} \\ (11^2)^{15} &\equiv 121^{15} \pmod{13} \\ 121^{15} &\equiv 4^{15} \pmod{13} \quad (4 \text{ is remainder of } 121/13) \\ 4^{15} &\equiv (4^3)^5 \pmod{13} \\ (4^3)^5 &\equiv 64^5 \pmod{13} \\ 64^5 &\equiv (-1)^5 \pmod{13} \quad (64 \text{ is 1 less than } 65, 65 = 5 \cdot 13) \\ (-1)^5 &\equiv -1 \pmod{13} \end{aligned}$$

Hence, $p = 13$ is the smallest odd prime factor.

Problem 5

Solve the following modular system:

$$\begin{aligned} x &\equiv 3 \pmod 8 \\ x &\equiv 2 \pmod 5 \\ x &\equiv 9 \pmod{11} \end{aligned}$$

Solution 5

By the application of Chinese Remainder solution our solution $x = x_p + M \cdot c$ where $M \in \mathbb{Z}$ and c is a specific value once added to a particular solution does not change the congruences.
We have $c = \text{LCM}(8, 5, 11) = 440$ as it does not change the congruence is because they are all divisible by the bases such that it only adds a 0 to each equation.
Now to find x_p.
$x_p = (440/8) \cdot 3 \cdot y_1 + (440/5) \cdot 2 \cdot y_2 + (440/11) \cdot 9 \cdot y_3$ where y_1, y_2, y_3 are modular inverses such that $\frac{440}{8} y_1 \equiv 1 \pmod 8$, $\frac{440}{5} y_2 \equiv 1 \pmod 5$, $\frac{440}{11} y_3 \equiv 1 \pmod{11}$. To find these modular inverses:

- For y_1, $55y_1 \equiv 1 \pmod 8$. Note that $55 = 56 - 1 = 8 \cdot 7 - 1$ which means $55 \equiv -1 \pmod 8$ such that $-y_1 \equiv 1 \pmod 8 \leftrightarrow y_1 \equiv -1 \pmod 8$ hence $y_1 = 7$ is valid.
- For y_2, $88y_1 \equiv 1 \pmod 5$. Note that $88 = 85 + 3 = 5 \cdot 17 + 3$ which means $88 \equiv 3 \pmod 5$ such that $3y_2 \equiv 1 \pmod 5$ hence $y_2 = 2$ is a suitable solution by inspection as $3 \cdot 2 = 6 = 5 \cdot 1 + 1$.
- For y_3, $40y_3 \equiv 1 \pmod{11}$. Note that $40 = 11 \cdot 3 + 7$ which means $40 \equiv 7 \pmod{11}$ such that $7y_3 \equiv 1 \pmod{11}$ hence by trial and error $y_3 = 8$ would be a suitable solution as $7 \cdot 8 = 56 = 11 \cdot 5 + 1$.

Hence $x_p = (440/8) \cdot 3 \cdot 7 + (440/5) \cdot 2 \cdot 2 + (440/5) \cdot 9 \cdot 8 = 55 \cdot 21 + 88 \cdot 4 + 40 \cdot 72 = 4387$ which means our final answer is $x = 4387 + 440 \cdot M_1 = 427 + 440 \cdot M$ where $M = 9 + M_1 \in \mathbb{Z}$ or $x \equiv 427 \pmod{440}$.

This question is meant to be beginner friendly which is why a lot of working out is presented in a simple manner. In general to solve a system $x \equiv a_i \pmod{k_i}$ for $i = 1$ to k if we define $K = \text{LCM}(k_1, ...k, n)$, $x \equiv \sum_{i=1}^{n} (K/k_i) a_i y_i \pmod K$ where $(K/k_i) y_i \equiv 1 \pmod{k_i}$.

Problem 6

Prove that for every positive integer n, there exists n consecutive positive integers such that none of them is a power of a prime.

Solution 6

We show that for any positive integer n, we can construct a number x such that none of $x + 1, x + 2, ..., x + n$ are perfect powers of a prime.
Fix distinct primes $p_1, p_2 ... p_n, q_1, q_2, ..., q_n$. Consider the following system of congru-

ences:

$$\begin{aligned} x+1 &\equiv 0 \pmod{p_1q_1} \\ x+2 &\equiv 0 \pmod{p_2q_2} \\ &\vdots \\ x+n &\equiv 0 \pmod{p_nq_n} \end{aligned}$$

Rearranging, we can construct the following equations:

$$\begin{aligned} x &\equiv -1 \pmod{p_1q_1} \\ x &\equiv -2 \pmod{p_2q_2} \\ &\vdots \\ x &\equiv -n \pmod{p_nq_n} \end{aligned}$$

By Chinese Remainder Theorem, we can find a $x \pmod{p_1p_2...p_nq_1q_2...q_n}$ which satisfies the equations above. Note that this means each of $x+1, x+2, ..., x+n$ is divisible by at least two primes, meaning that we have formed n consecutive integers such that none is a prime power.
Scratch: The trick to solving this question is to construct a scenario where distinct integers are products of distinct primes instead such that you can form a modular system, bringing into the form of Chinese Remainder Theorem's (CRT) desired form, call upon CRT to confirm the positive integer that this consecutive integer sequence is built up on exists. This question was shown in IMO 1989.

Solution 7

Baby RSA 1

Follow the RSA decryption algorithm. Use the following python solve script.

```
from Crypto.Util.number import long_to_bytes

# Paste the values printed from the first script here
N = int(input("N: "))
p = int(input("p: "))
q = int(input("q: "))
c = int(input("c: "))

# ========================
# Decrypt
# ========================
phi = (p - 1) * (q - 1)
d = pow(65537, -1, phi)
m = pow(c, d, N)
plaintext = long_to_bytes(m)
```

```
print("=== GIVEN VALUES ===")
print(f"N = {N}")
print(f"phi = {phi}")
print(f"d = {d}")
print(f"c = {c}")

print("\n=== DECRYPTED RESULT ===")
print(f"message_as_int = {m}")
print()
print(f"plaintext = {plaintext}")
```

Solution 8

Baby RSA 2

Observe that $(p-1)(q-1) = pq - p - q + 1 = pq - (p+1) + 1$. Using this, we can compute $\phi(N)$ from knowing $N = pq$ and $p + q$.

```
from Crypto.Util.number import long_to_bytes

# Paste the values printed from the first script here
N = int(input("N: "))
pq = int(input("p+q: "))
c = int(input("c: "))

# ========================
# Decrypt
# ========================
phi = N - pq + 1
d = pow(65537, -1, phi)

m = pow(c, d, N)
plaintext = long_to_bytes(m)

print("=== GIVEN VALUES ===")
print(f"N = {N}")
print(f"phi = {phi}")
print(f"d = {d}")
print(f"c = {c}")

print("\n=== DECRYPTED RESULT ===")
print(f"message_as_int = {m}")
print()
print(f"plaintext = {plaintext}")
```

Solution 9

common mod

Given $m^{e_1}, m^(e_2)$, encrypted under the same mod N, we can find values for (a, b) such that $a * e_1 + b * e2 = 1$. Using this, we can then calculate $c_1^a \cdot c_2^b$ to get $m^{a \cdot e_1 + b \cdot e_2} = m^1$. This allows us to recover the original message.

```
from Crypto.Util.number import long_to_bytes, inverse

# Paste the printed values from the challenge here
N = int(input("N: "))
e1 = int(input("e1: "))
e2 = int(input("e2: "))
c1 = int(input("c1: "))
c2 = int(input("c2: "))

# Extended Euclidean Algorithm
# This is where we find Bézout coefficients a, b such that:
#     a*e1 + b*e2 = gcd(e1, e2)
# Since gcd(e1, e2) = 1 in this challenge,
# we get:
#     a*e1 + b*e2 = 1
def egcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = egcd(b, a % b)
    x = y1
    y = x1 - (a // b) * y1
    return g, x, y

g, a, b = egcd(e1, e2)
assert g == 1

print("[+] Bézout coefficients found:")
print(f"    a = {a}")
print(f"    b = {b}")
print(f"    check: a*e1 + b*e2 = {a*e1 + b*e2}")

# We know:
#   c1 = m^e1 mod N
#   c2 = m^e2 mod N
#
# Using Bézout:
#   a*e1 + b*e2 = 1
#
# So:
#   m = m^1 = m^(a*e1 + b*e2)
#     = (m^e1)^a * (m^e2)^b
#     = c1^a * c2^b mod N
```

```
#
# If a or b is negative, we use modular inverse.

if a < 0:
    part1 = pow(inverse(c1, N), -a, N)
else:
    part1 = pow(c1, a, N)

if b < 0:
    part2 = pow(inverse(c2, N), -b, N)
else:
    part2 = pow(c2, b, N)

m = (part1 * part2) % N

print("[+] Recovered bytes:")
print(long_to_bytes(m).decode())
```

Solution 10

halfstad attack

We don't quite have enough message for Hastad's Attack. Instead, notice that N_1, N_2 share a common factor, so we can *gcd* algorithm to factorise N_1 and decrypt.

```
from Crypto.Util.number import long_to_bytes

N1 = 154950829963764444457971030984383287334833404582652440298997221609615
N2 = 867463841726180184019982621333255456326396328795978417784040549758824

c1 = int(input("c1: "))

def gcd(a, b):
    if a == 0:
        return b
    return gcd(b % a, a)

p = gcd(N1, N2)
q = N1 // p

# ========================
# Decrypt
# ========================
phi = (p - 1) * (q - 1)
d = pow(3, -1, phi)
m = pow(c1, d, N1)
plaintext = long_to_bytes(m)
```

```
print("\n=== DECRYPTED RESULT ===")
print(f"message_as_int = {m}")
print()
print(f"plaintext = {plaintext}")
```

Solution 11

same prime

Since N is made from the two identical primes, we can recover the prime used by square root of N, then we can calculate $\phi(N)$ using $p \cdot (p-1)$ because the number of integers that are divisible by p is just p so $\phi(N) = p^2 - p = p \cdot (p-1)$.
Then we can recover d to decrypt the message by finding $d = e^{-1} \pmod{\phi(N)}$ using python.

```
# solve.py
from math import isqrt
from Crypto.Util.number import long_to_bytes

n = int(input("n: "))
e = int(input("e: "))
c = int(input("c: "))

p = isqrt(n)
assert p * p == n

phi = p * (p - 1)
d = pow(e, -1, phi)
m = pow(c, d, n)

print(long_to_bytes(m))
```